# Gotchas

To begin, we should make something about SymPy clear. SymPy is nothing more than a Python library, like `NumPy`, `Django`, or even modules in the Python standard library `sys` or `re`. What this means is that SymPy does not add anything to the Python language. Limitations that are inherent in the Python language are also inherent in SymPy. It also means that SymPy tries to use Python idioms whenever possible, making programming with SymPy easy for those already familiar with programming with Python. As a simple example, SymPy uses Python syntax to build expressions. Implicit multiplication (like `3x` or `3 x`) is not allowed in Python, and thus not allowed in SymPy. To multiply `3` and `x`, you must type `3*x` with the `*`.

# Symbols

One consequence of this fact is that SymPy can be used in any environment where Python is available. We just import it, like we would any other library:

```
>>> from sympy import *
```

This imports all the functions and classes from SymPy into our interactive Python session. Now, suppose we start to do a computation.

```
>>> x + 1
Traceback (most recent call last):
...
NameError: name 'x' is not defined
```

Oops! What happened here? We tried to use the variable `x`, but it tells us that `x` is not defined. In Python, variables have no meaning until they are defined. SymPy is no different. Unlike many symbolic manipulation systems you may have used, in SymPy, variables are not defined automatically. To define variables, we must use `symbols`.

```
>>> x = symbols('x')
>>> x + 1
x + 1
```

`symbols` takes a string of variable names separated by spaces or commas, and creates Symbols out of them. We can then assign these to variable names. Later, we will investigate some convenient ways we can work around this issue. For now, let us just define the most common variable names, `x`, `y`, and `z`, for use through the rest of this section

```
>>> x, y, z = symbols('x y z')
```

As a final note, we note that the name of a Symbol and the name of the variable it is assigned to need not have anything to do with one another.

```
>>> a, b = symbols('b a')
>>> a
b
>>> b
a
```

Here we have done the very confusing thing of assigning a Symbol with the name `a` to the variable `b`, and a Symbol of the name `b` to the variable `a`. Now the Python variable named `a` points to the SymPy Symbol named `b`, and vice versa. How confusing. We could have also done something like

```
>>> crazy = symbols('unrelated')
>>> crazy + 1
unrelated + 1
```

This also shows that Symbols can have names longer than one character if we want.

Usually, the best practice is to assign Symbols to Python variables of the same name, although there are exceptions: Symbol names can contain characters that are not allowed in Python variable names, or may just want to avoid typing long names by assigning Symbols with long names to single letter Python variables.

To avoid confusion, throughout this tutorial, Symbol names and Python variable names will always coincide. Furthermore, the word "Symbol" will refer to a SymPy Symbol and the word "variable" will refer to a Python variable.

Finally, let us be sure we understand the difference between SymPy Symbols and Python variables. Consider the following:

```
x = symbols('x')
expr = x + 1
x = 2
print(expr)
```

What do you think the output of this code will be? If you thought `3`, you're wrong. Let's see what really happens

```
>>> x = symbols('x')
>>> expr = x + 1
>>> x = 2
>>> print(expr)
x + 1
```

Changing `x` to `2` had no effect on `expr`. This is because `x = 2` changes the Python variable `x` to `2`, but has no effect on the SymPy Symbol `x`, which was what we used in creating `expr`. When we created `expr`, the Python variable `x` was a Symbol. After we created, it, we changed the Python variable `x` to 2. But `expr` remains the same. This behavior is not unique to SymPy. All Python programs work this way: if a variable is changed, expressions that were already created with that variable do not change automatically. For example

```
>>> x = 'abc'
>>> expr = x + 'def'
>>> expr
'abcdef'
>>> x = 'ABC'
>>> expr
'abcdef'
```

**Quick Tip**

To change the value of a Symbol in an expression, use `subs`

```
>>> x = symbols('x')
>>> expr = x + 1
>>> expr.subs(x, 2)
3
```

In this example, if we want to know what `expr` is with the new value of `x`, we need to reevaluate the code that created `expr`, namely, `expr = x + 1`. This can be complicated if several lines created `expr`. One advantage of using a symbolic computation system like SymPy is that we can build a symbolic representation for `expr`, and then substitute `x` with values. The correct way to do this in SymPy is to use `subs`, which will be discussed in more detail later.

```
>>> x = symbols('x')
>>> expr = x + 1
>>> expr.subs(x, 2)
3
```

# Equals signs

Another very important consequence of the fact that SymPy does not extend Python syntax is that `=` does not represent equality in SymPy. Rather it is Python variable assignment. This is hard-coded into the Python language, and SymPy makes no attempts to change that.

You may think, however, that `==`, which is used for equality testing in Python, is used for SymPy as equality. This is not quite correct either. Let us see what happens when we use `==`.

```
>>> x + 1 == 4
False
```

Instead of treating `x + 1 == 4` symbolically, we just got `False`. In SymPy, `==` represents exact structural equality testing. This means that `a == b` means that we are *asking* if $a = b$. We always get a `bool` as the result of `==`. There is a separate object, called `Eq`, which can be used to create symbolic equalities

```
>>> Eq(x + 1, 4)
Eq(x + 1, 4)
```

There is one additional caveat about `==` as well. Suppose we want to know if $(x + 1)^2 = x^2 + 2x + 1$. We might try something like this:

```
>>> (x + 1)**2 == x**2 + 2*x + 1
False
```

We got `False` again. However, $(x + 1)^2$ *does* equal $x^2 + 2x + 1$. What is going on here? Did we find a bug in SymPy, or is it just not powerful enough to recognize this basic algebraic fact?

Recall from above that `==` represents *exact* structural equality testing. "Exact" here means that two expressions will compare equal with `==` only if they are exactly equal structurally. Here, $(x + 1)^2$ and $x^2 + 2x + 1$ are not the same structurally. One is the power of an addition of two terms, and the other is the addition of three terms.

It turns out that when using SymPy as a library, having `==` test for exact structural equality is far more useful than having it represent symbolic equality, or having it test for mathematical equality. However, as a new user, you will probably care more about the latter two. We have already seen an alternative to representing equalities symbolically, `Eq`. To test if two things are equal, it is best to recall the basic fact that if $a = b$, then $a - b = 0$. Thus, the best way to check if $a = b$ is to take $a - b$ and simplify it, and see if it goes to 0. We will learn later that the function to do this is called `simplify`. This method is not infallible—in fact, it can be theoretically proven that it is impossible to determine if two symbolic expressions are identically equal in general—but for most common expressions, it works quite well.

```
>>> a = (x + 1)**2
>>> b = x**2 + 2*x + 1
>>> simplify(a - b)
0
>>> c = x**2 - 2*x + 1
>>> simplify(a - c)
4*x
```

There is also a method called `equals` that tests if two expressions are equal by evaluating them numerically at random points.

```
>>> a = cos(x)**2 - sin(x)**2
>>> b = cos(2*x)
>>> a.equals(b)
True
```

# Two Final Notes: `^` and `/`

You may have noticed that we have been using `**` for exponentiation instead of the standard `^`. That's because SymPy follows Python's conventions. In Python, `^` represents logical exclusive or. SymPy follows this convention:

```
>>> True ^ False
True
>>> True ^ True
False
>>> Xor(x, y)
x ^ y
```

Finally, a small technical discussion on how SymPy works is in order. When you type something like `x + 1`, the SymPy Symbol `x` is added to the Python int `1`. Python's operator rules then allow SymPy to tell Python that SymPy objects know how to be added to Python ints, and so `1` is automatically converted to the SymPy Integer object.

This sort of operator magic happens automatically behind the scenes, and you rarely need to even know that it is happening. However, there is one exception. Whenever you combine a SymPy object and a SymPy object, or a SymPy object and a Python object, you get a SymPy object, but whenever you combine two Python objects, SymPy never comes into play, and so you get a Python object.

```
>>> type(Integer(1) + 1)
<class 'sympy.core.numbers.Integer'>
>>> type(1 + 1)
<... 'int'>
```

This is usually not a big deal. Python ints work much the same as SymPy Integers, but there is one important exception: division. In SymPy, the division of two Integers gives a Rational:

```
>>> Integer(1)/Integer(3)
1/3
>>> type(Integer(1)/Integer(3))
<class 'sympy.core.numbers.Rational'>
```

But in Python `/` represents either integer division or floating point division, depending on whether you are in Python 2 or Python 3, and depending on whether or not you have run `from __future__ import division` in Python 2 which is no longer supported from versions above SymPy 1.5.1:

```
>>> from __future__ import division
>>> 1/2
0.5
```

To avoid this, we can construct the rational object explicitly

```
>>> Rational(1, 2)
1/2
```

This problem also comes up whenever we have a larger symbolic expression with `int/int` in it. For example:

```
>>> x + 1/2
x + 0.5
```

This happens because Python first evaluates `1/2` into `0.5`, and then that is cast into a SymPy type when it is added to `x`. Again, we can get around this by explicitly creating a Rational:

```
>>> x + Rational(1, 2)
x + 1/2
```

There are several tips on avoiding this situation in the Gotchas and Pitfalls document.

# Further Reading

For more discussion on the topics covered in this section, see Gotchas and Pitfalls.

Copyright © 2024 SymPy Development Team
Made with Sphinx and @pradyunsg's Furo
Last updated on Sep 18, 2024